

Architekturmuster

Ansätze für nachhaltige Softwarestrukturen

**Architekturmuster in der Softwareentwicklung | Architektur
pattern | Software Engineering**

<https://www.youtube.com/watch?v=OlafuLDsXgg>

Was versteht man unter Cargo-Kult?



<https://de.wikipedia.org/wiki/Cargo-Kult>



<https://blog.ppedv.de/post/cargo-cult-programming>

- "Cargo-Kult" stammt aus Melanesien und entstand Ende des 19. Jahrhunderts im Zuge des Kolonialismus.
- Verschiedene Nationen bauten im Pazifikraum strategische Stützpunkte auf und belieferten diese per Flugzeug mit wertvollen Gütern.
- Die Ureinwohner melanesischer Inseln (Neuguinea, Neukaledonien und den Salomonen) hatten bis dahin kaum Kontakt mit Technologie und ihnen fehlte das Verständnis dafür, was ein Flugzeug ist, wie es funktioniert und warum es Güter ("Cargo") abwirft.
- Infolgedessen entstand ein Kult um westliche Frachtgüter -- der sog. Cargo Cult.
- Nach dem Abzug der Truppen versuchten die Inselbewohner, diese Güter durch die Nachahmung der beobachteten Handlungen und Rituale wieder herbeizubeschwören.
- Der Cargo-Kult erlebte einen bedeutenden Aufschwung während des Zweiten Weltkriegs.

<https://www.dev-insider.de/cargo-cult-programming-was-ist-das-a-bfd5af688ebe1c917dafcd92b24e812b/>

Gefahr "Cargo Cult Programming"

Definition

"Cargo Culted" bezieht sich auf unreflektierte Übernahme von Praktiken oder Strukturen ohne Verständnis.

- Blinde Anwendung von Schichten ohne Anpassung.
- Übernahme erfolgreicher Muster ohne eigene Analyse.

<https://blog.ppedv.de/post/cargo-cult-programming>

Resultat

- Unnötige Komplexität und ineffiziente Kommunikation
- Schwer wartbarer Code

Vermeidung

- Verständnis der Architekturprinzipien
- Kritische Bewertung und Anpassung an Projektanforderungen

<https://www.dev-insider.de/cargo-cult-programming-was-ist-das-a-bfd5af688ebe1c917dafcd92b24e812b/>

Beispiele für Cargo Cult Programming

- **Kopieren, ohne zu verstehen:** Entwickler übernehmen Beispiele 1:1, ohne zu wissen, warum der Lösungsweg gewählt wurde und weshalb er funktioniert, z. B. Vorschläge aus Foren.
- **Kopieren, ohne anzupassen:** Entwickler übernehmen Standardlösungen aus Dokumentationen oder Fachliteratur, ohne diese für den aktuellen Fall anzupassen.
- **Blinder Einsatz von immer gleichen Mitteln:** Entwickler nutzen immer die gleichen Muster oder Frameworks mit der Vorstellung, dass diese notwendig sind, weil sie auch in anderen Situationen erfolgreich waren.
- **Überflüssiger Code:** Entwickler fügen Code ein, der zum Lösen des Problems nicht notwendig ist. Weil der Code auf exakt diese Art in einem anderen Projekt mit zusätzlichen Anforderungen erfolgreich benutzt wurde, wird er als Teil der Lösung wahrgenommen, z. B. Optimierungsmaßnahmen oder Zusatzfunktionen.

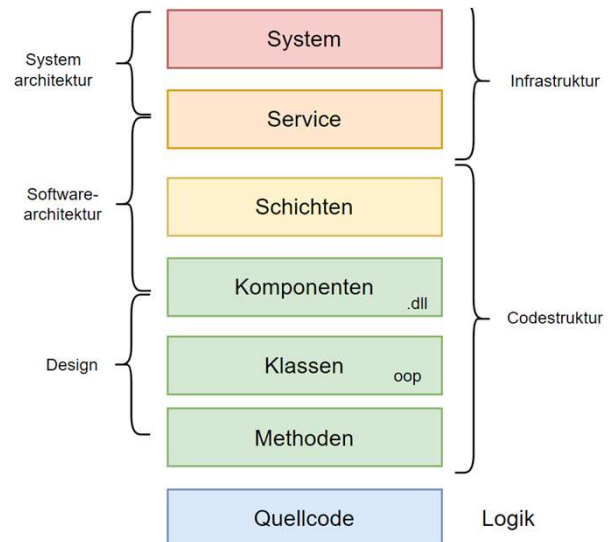
Welche Probleme können durch Cargo Cult Programming entstehen?

- schlechte Qualität durch ineffizienten, falschen, unsicheren oder schlecht wartbaren Code

- schwere Fehler durch unerkannte Wechselwirkungen des falschen Codes
- mangelhafte Ergebnisse durch bloßes Übernehmen ohne Anpassung an die aktuellen Erfordernisse
- schwere Kommunikation mit anderen Entwicklern oder Teams durch fehlendes Verständnis des Codes
- hohe Komplexität durch unnötigen Code

Hierarchien von Patterns

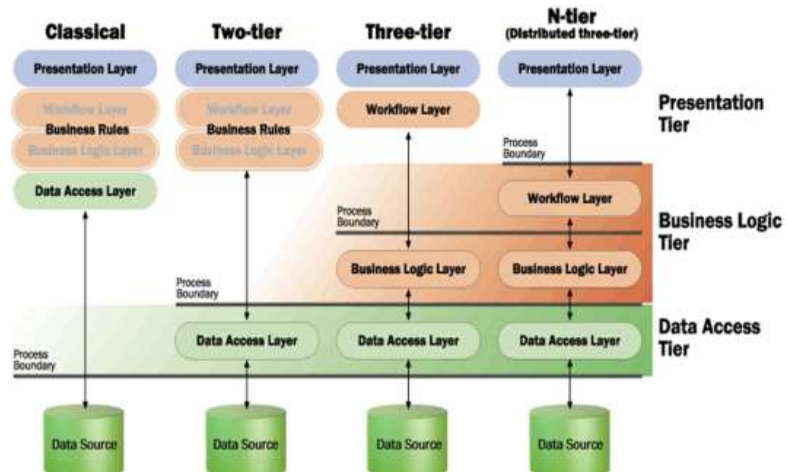
- Systemmuster
 - Monolith, SOA / Microservices, Client-Server uvm.
- Architekturmuster
 - N-Tier, Onion, Hexagonal, Clean, Vertical uvm.
- Entwurfsmuster (Design Patterns)
 - Erzeugermuster
 - Strukturmuster
 - Verhaltensmuster



SOA / Microservices: Service-orientierte Architektur (SOA) und Microservices sind Ansätze zur Strukturierung von Systemen auf einer hohen Abstraktionsebene. Sie helfen, große Systeme in kleinere, unabhängige Dienste zu unterteilen.

N-Tier / Layer – Schichtenarchitektur (horizontal)

- Logische, manchmal physische Trennung
- Layer bildet logische Struktur der App ab
- Tier bildet physische Struktur des Systems ab
- Begriffe Tier / Layer sind jedoch austauschbar



<https://de.wikipedia.org/wiki/Schichtenarchitektur>

[Business logic - Wikipedia](https://de.wikipedia.org/wiki/Business_logic)

<https://nileshviradiya.blogspot.com/2014/01/what-is-meant-by-n-tier.html>

https://files.gotocon.com/uploads/slides/conference_12/515/original/gotoberlin2018-modular-monoliths.pdf

Vielleicht ist es das am weitesten verbreitete, es ist überall zu finden. Fast alle Anwendungen sind um sie herum aufgebaut. Es ist leicht zu verstehen und gut zu beginnen.

Bei diesem Muster organisieren wir in horizontalen Schichten. Die Anzahl der Schichten kann von einer Anwendung zur anderen unterschiedlich sein. Im Allgemeinen haben wir:

- Die Webschicht: verarbeitet eingehende Anfragen, rendert HTML, usw.).
- Die Geschäftsschicht: setzt die Geschäftsregeln für die Daten durch (der Kunde kann nicht von einem leeren Konto abheben)
- Die Datenschicht: Sie stellt eine Verbindung zur Datenbank her, um die Daten abzufragen und zu aktualisieren. Diese werden oft als Datenzugriffsobjekte (DAO) bezeichnet.

Wenn die Anfrage eintrifft, wird sie von oben nach unten durchlaufen.

Vorteile Schichtenarchitektur

- Verbesserte Wartbarkeit und Erweiterbarkeit
- Klare Trennung von Verantwortlichkeiten
- Förderung der Modularität und Wiederverwendbarkeit
- Parallele Entwicklung durch verschiedene Teams möglich
- Verbesserte Sicherheit durch Trennung der Datenzugriffsschicht

Vorteile von Schichten / Tiers in der Softwarearchitektur

Warum Schichtenarchitektur?

- Verbesserte Wartbarkeit und Erweiterbarkeit[1][2]
- Klare Trennung von Verantwortlichkeiten[2]
- Unabhängige Entwicklung und Tests der Schichten[1][3]
- Bessere Skalierbarkeit und Performance[1][3]
- Einfacherer Austausch einzelner Schichten[1]
- Förderung der Modularität und Wiederverwendbarkeit[2]
- Parallele Entwicklung durch verschiedene Teams möglich[2]
- Vereinfachte Integration neuer Funktionen[4]
- Verbesserte Sicherheit durch Trennung der Datenzugriffsschicht[4]

Beispiel: Dreischichtige Architektur

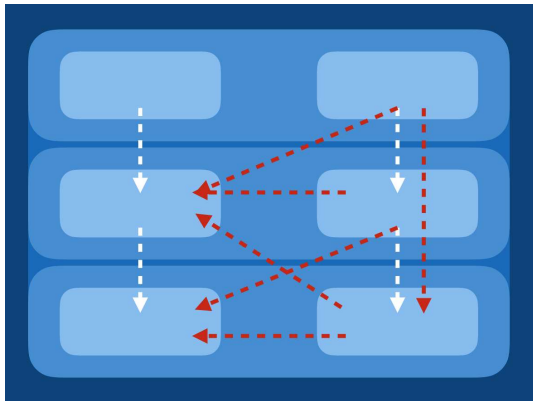
1. Präsentationsschicht (UI)
2. Anwendungslogikschicht (Geschäftslogik)
3. Datenschicht (Datenbank)

Diese Struktur ermöglicht eine flexible, wartbare und skalierbare Softwarearchitektur.

Citations:

- [1] <https://www.johner-institut.de/blog/systems-engineering/schichtenarchitektur/>
- [2] <https://www.studysmarter.de/ausbildung/ausbildung-in-it/fachinformatiker-anwendungsentwicklung/schichtenarchitektur/>
- [3] <https://www.ibm.com/de-de/topics/three-tier-architecture>
- [4] <https://insightsoftware.com/de/blog/5-benefits-of-a-3-tier-architecture/>
- [5] <https://www.heise.de/blog/Patterns-in-der-Softwarearchitektur-Das-Schichtenmuster-7941983.html>

"Caveats" der Schichtenarchitektur



- Wird Realität abgebildet?
- Spiegelt es den Code?
- Jede Schicht hübsch und sauber getrennt?
- Constraints nur "soft" in Dokumentation?

<https://medium.com/omarelgabrys-blog/component-based-architecture-3c3c23c7e348>

Aber warum? Trennung von Belangen, Entkopplung, oder vielleicht, weil uns gesagt wurde, dass dies eine gute Sache ist.

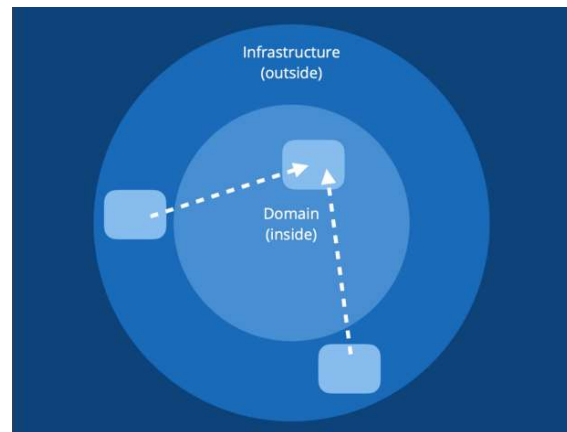
Nun, das könnte sein. Aber die Frage hier ist, ob die Architekturdesigndiagramme den Code widerspiegeln. Wir denken an Komponenten, aber wir schreiben eine schichtweise Architektur. Diese Komponenten gibt es im Code nicht, und die Programmiersprache hat auch kein Schlüsselwort für Schichten oder Komponenten.

Und ruft jede Schicht nur die darunter liegende Schicht auf oder kann sie auch übersprungen werden? Bleibt jede Schicht nett und sauber? Der Controller ruft manchmal das Modell auf, und das Modell ruft die Dienste zurück; die Aufrufe sind überall, da sie öffentlich sind.

Die Beschränkungen, die wir in der Dokumentation angeben oder den Entwicklern mitteilen, können nicht durchgesetzt werden. Wir sind vielleicht in der Lage, Code-Reviews durchzuführen. Aber wir sind alle Menschen, und um schneller zu liefern, könnten wir gezwungen oder versucht sein, den schlechten Code trotzdem zu akzeptieren.

Ports-und-Adapter-Architektur (Onion/Hexagonal)

- Domäne mit Geschäftslogik innen
- Datenaustausch über abstrakte Ports (Input / Output)
- Konkrete Implementierung nach außen hin mittels Adapter
- Kann log. Mapping "erschweren"



https://de.wikipedia.org/wiki/Hexagonale_Architektur

<https://medium.com/omarelgabrys-blog/component-based-architecture-3c3c23c7e348>

<https://www.youtube.com/watch?v=9N1LGp6Xfho>

Hauptunterschied zu Layer / Tiers: **Stärkere Betonung auf Schnittstellen und Abstraktionen**

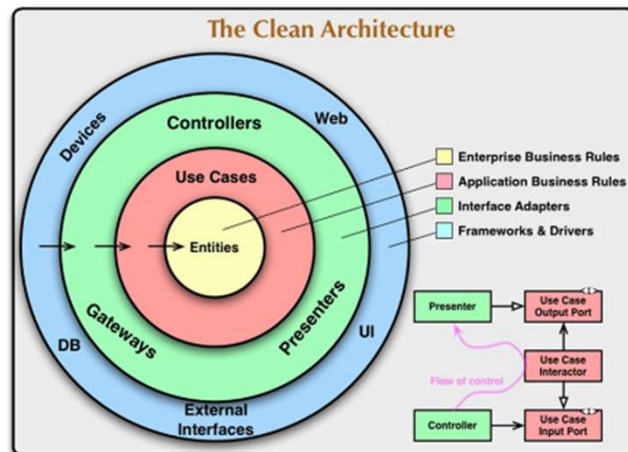
Der Ausdruck „hexagonal“ unterstellt, dass es sechs Teile des Konzepts geben muss, während es in Wirklichkeit nur vier Schlüsselfelder gibt: Benachrichtigung, Persistenz (Speicherung), Geschäftsereignisse, Verwaltung.

Nach [Martin Fowler](#) hat die Sechseck-Architektur den Vorteil, Ähnlichkeiten zwischen Präsentationsschicht und Datenquellschicht zu nutzen, um symmetrische Komponenten zu schaffen, die aus einem Kern und einem Ring an Schnittstellen gebaut wurden, **doch mit dem Nachteil**, dass die innewohnende, natürliche Asymmetrie zwischen Dienstanbietern und Dienstbenutzern verborgen wird, die durch Schichten besser repräsentiert werden würden.^[3] Beispielsweise kann der Dienstbenutzer nie mehr Daten verarbeiten, als der Dienstanbieter bereitstellt, während der Dienstanbieter durch viel umfangreicheres Wissen grundsätzlich in der Lage ist, nicht nur die vom Dienstbenutzer benötigten, sondern all seine Daten in beliebiger Form bereitzustellen. In einer Sechseck-Architektur

wird unterstellt, dass sich die Komponenten ihr Wissen und ihre Daten gleichberechtigt teilen.

The Clean Architecture

- Proposal von Robert C. Martin aka "Uncle Bob"
- Abgeleitet und inspiriert von
 - Onion Architektur
 - Hexagonal Architektur
- Domänenzentrierter Ansatz zur Organisation von Abhängigkeiten



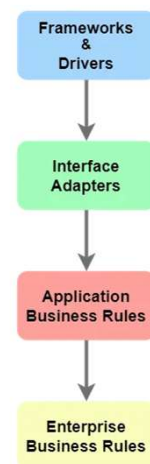
<https://betterprogramming.pub/the-clean-architecture-beginners-guide-e4b7058c1165> >> <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

In dem Diagramm sind exemplarisch 4 Ebenen zu sehen:
Blau, grün, rot, gelb

Jeder Kreis steht für einen anderen Bereich der Software.
Die äußerste Schicht ist die unterste Ebene der Software, und je tiefer wir vordringen, desto höher wird die Ebene.
Je tiefer wir einsteigen, desto weniger anfällig ist die Ebene für Änderungen.

Vorteile der Clean Architecture

- Klare Strukturierung
- Fokus auf Entitäten
- Flexible Schichtenanzahl
- Explizite Behandlung von Framework und Tools
- Betonung der Testbarkeit durch isoliertes Business Layer
- Detaillierte Prinzipien nach SOLID, insbesondere das DIP!



Die Clean Architecture von Uncle Bob (Robert C. Martin) bietet einige spezifische Vorteile und Alleinstellungsmerkmale im Vergleich zur Onion/Hexagonal Architektur:

- 1. Klarere Strukturierung:** Die Clean Architecture definiert explizit vier Hauptschichten (Entitäten, Anwendungsfälle, Schnittstellenadapter und Frameworks), was eine präzisere Strukturierung des Systems ermöglicht.
- 2. Fokus auf Entitäten:** Im Kern der Clean Architecture befinden sich Entitäten, die unternehmensweit gültige Geschäftsregeln repräsentieren. Dies betont die Wichtigkeit der Kerngeschäftslogik.
- 3. Flexibilität in der Schichtenanzahl:** Die Clean Architecture erlaubt eine flexible Anzahl von Schichten, abhängig von der Projektgröße, was eine bessere Anpassung an verschiedene Projektanforderungen ermöglicht.
- 4. Explizite Behandlung von Frameworks:** Die Clean Architecture positioniert Frameworks und externe Tools explizit in der äußersten Schicht, was die Unabhängigkeit des Systemkerns von externen Technologien unterstreicht.
- 5. Betonung der Testbarkeit:** Uncle Bob legt besonderen Wert auf

die verbesserte Testbarkeit durch die Isolation der Geschäftslogik, was in der Clean Architecture stärker hervorgehoben wird.

- 6. Detailliertere Prinzipien:** Die Clean Architecture beinhaltet spezifische Prinzipien nach SOLID, die in der Onion/Hexagonal Architektur nicht explizit erwähnt werden. SRP, OCP, LSP, ISP, DIP

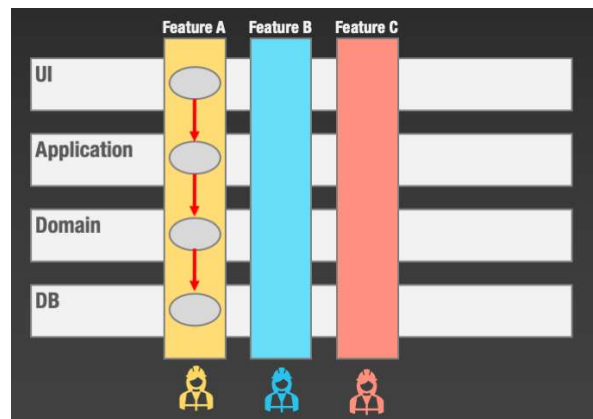
Trotz dieser Unterschiede teilen alle drei Architekturen das grundlegende Ziel, die Geschäftslogik von technischen Details zu entkoppeln und eine flexible, wartbare Softwarestruktur zu schaffen.

<https://alansantos.dev/2019/03/architecture/the-clean-architecture/>

<https://www.studysmarter.de/ausbildung/ausbildung-in-it/fachinformatiker-anwendungsentwicklung/clean-architecture/>

Vertical Slice Architecture (Feature Based)

- Vertikale Slices statt horizontaler Schichten
- Jedes Slice repräsentiert vollständiges Feature oder Use Case
- Unabhängige Entwicklung von Features
- Minimale Kopplung zw. Slices, maximale Kohäsion im Slice
- Fokus auf Geschäftswert



<https://www.predic8.de/vertical-slice-architecture.htm>

Warum? (Motivation)

Weil Änderungen häufig über alle Schichten hinweg bis zur DB angepasst werden müssen, wodurch sich Logik womöglich über die Schichten verteilt.

Vorteile

- Verbesserte Wartbarkeit und Erweiterbarkeit
- Einfachere Implementierung neuer Funktionen
- Reduzierte Abhängigkeiten zwischen Teams
- Fokus auf Geschäftsanforderungen statt technischer Schichten
- Flexibilität bei der Implementierung einzelner Features

Umsetzung

- Jedes Feature enthält alle notwendigen Komponenten (UI, Logik, Daten)
- Gemeinsam genutzte Funktionalitäten können in Services ausgelagert werden
- Einsatz von Mediatoren zur Entkopplung von Anfragen und Handlern
- Separate Datenhaltung pro Feature, Zugriff über definierte Schnittstellen

Diese Architektur ermöglicht eine effiziente, kundenorientierte Entwicklung mit schnellen Iterationen und klarem Fokus auf Geschäftswert.

Vergleich mit Microservices

Die *Vertical Slice Architecture* weist auf den ersten Blick Ähnlichkeiten mit der Microservices-Architektur auf, unterscheidet sich jedoch grundlegend in einigen Aspekten. Ein Microservice besitzt eine eigene Codebasis und kann unabhängig von anderen Services entwickelt, installiert, bereitgestellt und betrieben werden. Im Gegensatz dazu ist der Code eines Slice in der *Vertical Slice Architecture* zwar von den anderen Slices getrennt, jedoch sind die einzelnen Features alle Teil eines einzigen Projekts und werden gemeinsam bereitgestellt und betrieben.

Code Sharing

Fachklassen können in einem Model gekapselt werden, das von mehreren Slices verwendet wird. Funktionalitäten, die keinem einzelnen Use Case zugeordnet werden können, lassen sich in einen Service auslagern. Je nach Grad der Auslagerung in Services nähert sich die Architektur im Extremfall wieder der Schichtenarchitektur an.

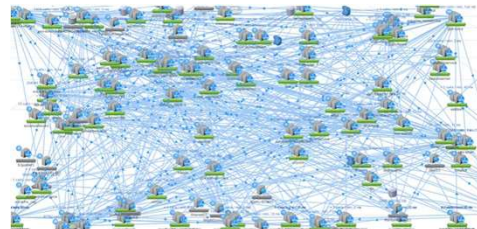
Vertical Slice Architecture (Feature Based)

Vorteile

- Konzentration auf Fachlichkeit erleichtert Entwicklung
- Einfache Verständlichkeit
- Unabhängige Teams
- Lose Kopplung

Nachteile

- Gefahr der geringen Abstraktion
- Komplexität der Gesamtarchitektur



[Netflix Mid-Tier Services](#)

Die Vertical Slice Architecture bietet eine Reihe von Vorzügen:

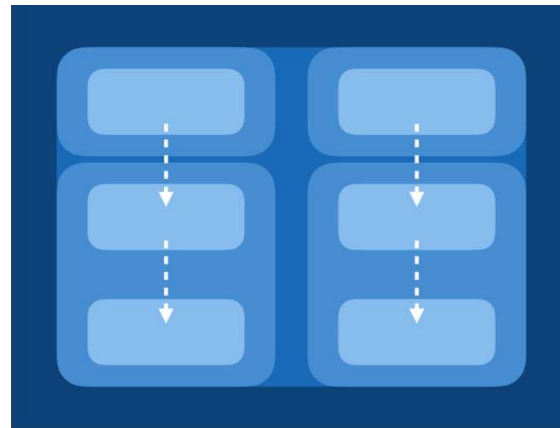
- Konzentration auf die Fachlichkeit: Durch die Organisation nach fachlichen Slices wird die Entwicklung und Wartung erleichtert. Änderungen oder Erweiterungen betreffen in der Regel nur eine begrenzte Anzahl von Dateien oder Verzeichnissen, wodurch Seiteneffekte minimiert werden.
- Bessere Testbarkeit: Die Isolation der Slices ermöglicht auch das getrennte Testen einzelner Funktionen und Use Cases
- Kürzere Einarbeitungszeit: Im Vergleich zu komplexeren Architekturen wie der hexagonalen oder Clean Architecture ist die Vertical Slice Architecture einfacher zu verstehen und erfordert weniger Zeit für die Einarbeitung. Entwickler können schneller produktiv werden, da sie sich auf die spezifischen Anforderungen und Implementierungsdetails innerhalb eines Slices konzentrieren können.
- Unabhängige Teams: Jedes Slice kann von einem dedizierten Team bearbeitet werden, ohne dass Abstimmungen mit anderen Teams erforderlich sind. Dies fördert die Autonomie und Flexibilität der Entwicklungsteams.
- Lose Kopplung: Die Vertical Slice Architecture minimiert die Abhängigkeiten zwischen verschiedenen Slices. Jedes Slice ist weitgehend unabhängig von anderen, was die Skalierbarkeit und Wartbarkeit der Anwendung verbessert.

Die Vertical Slice Architecture hat aber auch Nachteile:

- Gefahr der geringen Abstraktion: Es besteht das Risiko, dass zu wenig abstrahiert wird. Dies kann dazu führen, dass ähnliche Funktionalitäten in verschiedenen Slices dupliziert werden, was die Wartbarkeit und die Wiederverwendbarkeit des Codes beeinträchtigen kann.
- Komplexität der Gesamtarchitektur: Wenn die Anwendung wächst und viele Slices hinzugefügt werden, kann die Gesamtarchitektur komplexer werden.

Komponentenbasierte Architektur

- Hybrid zwischen horizontalem und feature-basiertem vertikalen Ansatz
- Eigenständige, modulare Einheiten mit spez. Funktionalität
- Fördert Modularität, lose Kopplung und hohe Kohäsion
- Effiziente Entwicklung



https://de.wikipedia.org/wiki/Komponentenbasierte_Entwicklung

<https://medium.com/omarelgabrys-blog/component-based-architecture-3c3c23c7e348>

Who is using it?

I don't know.

Oh, [React](#). It is a Component-Based JavaScript library for building UI.

Anyone else? ...

Komponentenbasierte Architektur

Vorteile

- Hohe Wiederverwendbarkeit
- Schnellere Entwicklung durch vorgefertigte Komponenten
- Verbesserte Wartbarkeit und Erweiterbarkeit
- Parallele Entwicklung verschiedener Komponenten möglich
- Gute Testbarkeit

Nachteile

- Herausforderungen bei der Integration verschiedener Komponenten
- Mögliche Kompatibilitätsprobleme zwischen Komponenten
- Abhängigkeitsmanagement kann komplex werden
- Potenzielle Performanceeinbußen durch Komponentenkommunikation
- Erfordert sorgfältige Planung und Dokumentation der Schnittstellen

<https://www.studysmarter.de/studium/ingenieurwissenschaften/automatisierung-in-der-informationstechnologie/komponentenbasierte-entwicklung/>

Modular Monolith

- Ein Deployment, aber intern in fachliche Module strukturiert
- Module sind hoch kohäsiv und lose gekoppelt
- Kommunikation erfolgt über klar definierte Schnittstellen
- Vermeidet die Komplexität verteilter Systeme
- Wird oft als „Goldilocks“-Architektur zwischen Monolith und Microservices beschrieben



<https://ardalis.com/introducing-modular-monoliths-goldilocks-architecture/>

<https://ardalis.com/introducing-modular-monoliths-goldilocks-architecture/>

<https://www.youtube.com/live/wkAc6K09pKQ?t=147s>

Modular Monolith

Vorteile

- Einfaches Deployment als einzelne Anwendung
- Geringere Komplexität als verteilte Systeme
- Klare Struktur durch fachliche Module
- Keine Netzwerk- oder Infrastrukturabhängigkeiten zwischen Modulen
- Gute Grundlage für spätere Aufteilung in Microservices

Nachteile

- Disziplin erforderlich, um Modulgrenzen sauber einzuhalten
- Technisch keine harte Trennung wie bei verteilten Systemen
- Gefahr von unerwünschten Abhängigkeiten zwischen Modulen
- Skalierung nur auf Anwendungsebene möglich
- Kann mit wachsender Größe wieder zum klassischen Monolith werden

.NET Architecture Guides

Kostenlose **.NET Architecture Guides** als E-Books bei Microsoft

- Microservice architecture e-book ([PDF](#))
- ASP.NET Core architecture e-book ([PDF](#))
- Cloud-native e-book ([PDF](#))
- Blazor e-book ([PDF](#))
- Enterprise Application Patterns Using .NET MAUI ([PDF](#))
- Azure quick start e-book ([PDF](#))

<https://dotnet.microsoft.com/en-us/learn/dotnet/architecture-guides>